

Zero-TTT

AlphaZero 与端到端测试时训练融合项目规划

个人研究与娱乐项目

2026 年 8 月

摘要

本文规划一个面向 19 路围棋的 AlphaZero 与 Test-Time Training (TTT) 融合项目。模型使用标准可二阶求导的空间注意力、AlphaGo Zero 风格的八局面输入，以及可在真实落子后更新的快权重记忆。快权重被视为由搜索监督的可学习状态，而不是对 361 个空间 token 的显式键值存储：长期块在整盘棋中保存计划与搜索纠偏，短期块在每个真实回合内吸收 MCTS 分支揭示的战术信息。系统分别学习长期与短期初始化；每盘棋开始时，黑白双方派生出两份相互隔离的长期状态，每手另为当前行棋方创建一份用后即弃的短期状态。项目以验证机制、完成训练闭环和保证可恢复性为目标，不安排消融实验、超参数搜索或棋力指标分析，也不维护多 batch 下不同样本的 TTT 状态。

棋盘与规则

19×19，贴目 7.5，面积计分，362 个动作

序列批量

固定 batch size 为 1；允许跨序列梯度累积

局部上下文

当前局面加过去 7 个局面，共 8 个局面

长期记忆

一份长期初始化，黑白两份每局私有运行时状态

单手适应

一份短期初始化，每手执行 warm search 后派生临时状态

二阶窗口

长期状态保留最近 16 个真实回合的计算图；短期图当手结束

默认执行方式

PyTorch eager mode；暂不使用 `torch.compile`

主要算力目标

Linux + Docker + 单张 NVIDIA GPU；CPU 用于开发测试

目录

1	项目定位与理论判断	4
1.1	项目目标	4
1.2	可行性结论	4
2	围棋状态、动作与规则环境	5
2.1	固定问题定义	5
2.2	网络输入与输出	5
3	模型结构与双时间尺度初始化	6
3.1	参考 Transformer	6
3.2	两类可训练初始化与运行时状态	6
3.3	信息隔离边界	7
3.4	接口与生命周期摘要	7
4	真实回合协议	7
5	训练目标与元学习	8
5.1	内循环	8
5.2	外循环	9
6	显存控制、梯度截断与恢复	10
6.1	16 手截断元梯度	10
6.2	Activation Checkpoint	10
6.3	暂不编译模型	10
6.4	持久化检查点	10
7	监督预训练数据	11
7.1	数据来源	11
7.2	预训练顺序	11
8	MCTS 与自博弈设置	12
8.1	搜索基线	12
8.2	模拟次数随机化	12
9	工程组织与容器化	13
10	敏捷开发冲刺	13

11 测试与验收标准	14
11.1 规则与数据正确性	14
11.2 双初始化、生命周期与隐私	14
11.3 梯度、截断与恢复	15
12 风险与明确不做的事项	15
13 推荐实施顺序	16

1 项目定位与理论判断

1.1 项目目标

Zero-TTT 的目标不是复现 AlphaGo Zero 的最终棋力，也不是验证 TTT 在围棋上的统计显著收益，而是完成以下工程闭环：

- 在可靠的 19 路围棋规则环境上运行 AlphaZero 风格的策略-价值网络与 MCTS；
- 让模型在一盘棋内通过真实落子持续更新私有长期快权重，把超过八局面窗口的信息压缩到参数状态中；
- 让模型在单手内从 warm MCTS 的访问分布与备份价值学习，再用适应后的短期状态重新搜索；
- 在训练时对两类快权重更新求二阶梯度，使长期初始化学会记忆、短期初始化学会适应；
- 支持监督预训练、自博弈训练、梯度截断、检查点恢复和 Docker 迁移；
- 保持个人项目所需的可读性，避免多 batch、多设备和复杂分布式状态管理。

1.2 可行性结论

该思路在理论和工程上均可实现。原版 TTT 把一个可学习模型的参数作为 RNN 隐状态，并以训练视图、标签视图和查询视图构造逐层的 K/V/Q 重建任务[1]。TTT-E2E 则明确去掉逐层键值绑定，直接使用网络末端的下一 token 任务损失更新后段 MLP，并通过外循环元学习优化其初始化[2]。

本项目采用后一种端到端原则。因此，361 个棋盘交叉点不是依次写入的历史 token，而是同一时刻的空间集合；它们继续使用完整非因果注意力与二维 RoPE。TTT 的时间轴定义在更高层：整盘的真实回合序列，以及单手 warm MCTS 访问到的局面集合。快权重压缩的是搜索对网络判断所做的纠正，而不是原始棋盘内容本身。

本项目与语言模型 TTT-E2E 并不完全等价，主要差异如下：

- 语言模型使用自然发生的下一 token 作为内循环目标；本项目中，短期块使用 warm tree 的策略与备份价值，行棋方长期块使用 final root，观察方只能使用公开的实际动作。
- MCTS 是离散搜索过程，本项目不对 MCTS 本身求导。访问分布被视为停止梯度的策略改进目标，因此“端到端”只表示外循环梯度穿过网络和快权重更新。
- 训练时只精确传播长期状态最近 16 手的二阶梯度；短期状态不跨手。早期长期更新通过截断近似处理，所以实现结果不是整盘精确元梯度。

因此，本项目应被描述为“受 TTT-E2E 启发的、带私有在线参数记忆的 AlphaZero”，而不是原论文算法在围棋上的直接移植。是否提升棋力是开放问题，不作为项目完成条件。

2 围棋状态、动作与规则环境

2.1 固定问题定义

项目只支持 19×19 棋盘，动作空间固定为 361 个落点加 1 个停着 (pass)，共 362 个动作。默认贴目为 7.5，采用面积计分。MVP 不支持让子棋、其他贴目、日式数子、其他棋盘尺寸或认输。

规则实现采用固定版本的 PettingZoo go_v5[6]，并在其 Minigo Position 之上封装适合树搜索的不可变状态接口。项目适配层负责：

- 克隆或派生子状态、生成合法动作掩码、提交真实动作；
- 维护完整动作历史和最近八个棋盘局面；
- 在策略输出前屏蔽非法动作；
- 将终局结果统一转换为当前行棋方视角的 $z \in \{-1, +1\}$ 。

2.2 网络输入与输出

输入张量固定为

$$x_t \in \{0, 1\}^{1 \times 17 \times 19 \times 19}.$$

前 16 个平面表示当前及过去 7 个局面中双方棋子的位置，最后一个平面表示当前行棋方。开局不足八个局面时，缺失历史使用全零平面填充。该形式与 AlphaGo Zero 的历史局面表示保持一致[3]。

网络另接收一个非空间角色标记

$$r \in \{\text{self}, \text{opponent}\},$$

用来区分“用自己的搜索分布学习”与“根据对手公开动作学习”。角色标记通过小型嵌入加入全局 token，不改变 17 平面棋盘接口。

输出包括：

- 策略 logits $l_t \in \mathbb{R}^{1 \times 362}$ ，经合法动作掩码后归一化；
- 价值 $v_t \in [-1, 1]^{1 \times 1}$ ，表示当前行棋方的预期终局结果。
- 逐动作辅助预测 $q_t \in [-1, 1]^{1 \times 362}$ ，拟合 MCTS 对已访问合法边的备份均值；它只提供稠密写入梯度，不参与 PUCT。

棋盘 token 对应 361 个落点，停着动作由全局 token 产生。策略头与逐动作 Q 头共享空间特征，但参数和损失独立。所有入口必须断言 batch size 为 1。多个 warm 节点顺序前向并累积为一次短期梯度；梯度累积不能在一个 batch 中同时保存多组快权重。

3 模型结构与双时间尺度初始化

3.1 参考 Transformer

MVP 使用固定的小型参考配置，避免在项目早期引入参数搜索：

配置项	默认值
Transformer 块数	8
隐藏维度	256
注意力头数	8
FFN 隐藏维度	1024
空间 token	361 个交叉点 + 1 个全局 token
注意力	完整、非因果、标准缩放点积注意力
归一化	LayerNorm
dropout	0
快权重块	最后 2 个块

每个普通块由注意力和静态 FFN 构成。最后两个块额外增加自适应 FFN，与各自的静态 FFN 形成双分支。倒数第二个块是**长期块**，其运行时参数跨真实回合保留；最后一个块是**短期块**，其运行时参数只在当前手内有效。长期块位于短期块之前，因此单手战术适应能够查询已经形成的整盘记忆。静态分支保存稳定知识；注意力、嵌入、二维 RoPE、归一化、输出头和静态 FFN 都属于慢权重。

3.2 两类可训练初始化与运行时状态

系统学习两个职责不同的快权重初始化

$$\theta_{\text{game}} = \{\theta_j\}_{j \in \mathcal{F}_{\text{game}}}, \quad \theta_{\text{search}} = \{\theta_j\}_{j \in \mathcal{F}_{\text{search}}}.$$

其中 $\mathcal{F}_{\text{game}}$ 和 $\mathcal{F}_{\text{search}}$ 分别是长期块与短期块内自适应 FFN 的参数集合。两者都是模型参数的一部分，但不存在独立的可训练黑方或白方初始化。

每盘开始时，从长期初始化创建两个数值相同但对象独立的运行时状态：

$$\phi_0^B = \text{clone}(\theta_{\text{game}}), \quad \phi_0^W = \text{clone}(\theta_{\text{game}}).$$

每个真实回合开始时，只为当前行棋方创建一份短期状态

$$\psi_{t,0} = \text{clone}(\theta_{\text{search}}),$$

并在 warm MCTS 后将其更新为 $\psi_{t,1}$ 。真实落子后无条件销毁 $\psi_{t,1}$ ；下一手不会继承其数值。

这一设计表达了两种不同的归纳偏置： ϕ^B 与 ϕ^W 学习各自整盘经历， ψ_t 则针对单个局面的搜索证据快速适应。它具有三个重要性质：

1. 没有按玩家复制模型参数。优化器和普通模型检查点只维护一份 θ_{game} 与一份 θ_{search} 。
2. 长期运行时成本仍是两份。一旦 ϕ^B 与 ϕ^W 分化，为保证隐私与独立适应，必须保存两份长期张量。
3. 短期状态不会直接提交。战术结论通过 final root 的监督蒸馏进长期块，而不是把 $\psi_{t,1}$ 复制或合并进 ϕ_t^c 。

3.3 信息隔离边界

黑方搜索只能读取 ϕ^B 与当手短期状态，白方搜索只能读取 ϕ^W 与当手短期状态。一方的 warm/final 访问分布、边 Q、搜索树统计和短期梯度不得作为另一方的更新输入。观察方只接收公开状态与实际动作。这里的“隐私”是对局算法内部的信息隔离，并非密码学安全；集中式自博弈进程和离线学习器仍可保存完整轨迹用于外循环训练。

3.4 接口与生命周期摘要

接口	含义
<code>ModelOutput</code>	策略 logits、当前行棋方标量价值和 362 维逐动作 Q；PUCT 只消费前两项。
<code>GameFastState</code>	黑白各一份，从 θ_{game} 派生，整盘持久化，只在真实落子提交后更新。
<code>SearchFastState</code>	当前行棋方每手一份，从 θ_{search} 派生，只接受 warm-tree 写入，落子后销毁。
<code>SearchTrace</code>	根及筛选内部节点的 (s, π, Q, V, N) ；它是只读、停止梯度的搜索产物，不进入检查点。

4 真实回合协议

设第 t 手的当前行棋方为 c ，另一方为 $\bar{c}$ ，真实落子前状态为 s_t 。

warm MCTS 固定执行 64 次模拟，并产生一份只读的 `SearchTrace`。写入集定义为

$$\mathcal{D}_t^{\text{warm}} = \{\text{root}\} \cup \text{Top8}\{u : N_u \geq 8, u \neq \text{root}\},$$

其中内部节点按总访问数 N_u 排序。对每个节点 u ，根据子边访问数和备份均值构造

$$\pi_u(a) = \frac{N_u(a)}{\sum_b N_u(b)}, \quad V_u = \sum_a \pi_u(a) Q_u(a).$$

未访问边、非法动作和不存在的内部节点不进入 Q 损失。根节点无条件保留，因此数据不足时协议自然退化为只从根学习。

1. 从 θ_{search} 创建 $\psi_{t,0}$ 。warm search 全程冻结慢权重、 ϕ_t^c 和 $\psi_{t,0}$ ；所有假想分支共享同一版本。

2. 用 $\mathcal{D}_t^{\text{warm}}$ 顺序前向多个 batch-size-1 局面，按节点访问数归一化损失并累积梯度。执行一次无动量 SGD，只把 $\psi_{t,0}$ 更新为 $\psi_{t,1}$ 。
3. 短期状态版本变化后立即清除 warm tree；禁止在新权重下复用旧节点的先验、叶值或边统计。
4. 固定 $(\phi_t^c, \psi_{t,1})$ 执行 final MCTS。final 搜索仍按当前回合类型使用 128 或 800 次模拟，产生 π_t^F 、 Q_t^F 、 V_t^F 和动作 a_t 。
5. 当前回合外层损失评价适应后网络相对 final 搜索目标与终局结果的预测，从而训练短期初始化“会想”。MCTS 及节点筛选过程全部停止梯度。
6. 由真实规则环境验证并提交 a_t 。只有提交成功，才允许更新长期状态。
7. 为避免把临时参数直接写入长期记忆，丢弃 $\psi_{t,1}$ ，使用 $(\phi_t^c, \psi_{t,0})$ 在 s_t 上重新前向，并以 final root 的 (π_t^F, Q_t^F, V_t^F) 更新行棋方长期块。短期块、慢权重和输出头在该内循环中冻结。
8. 观察方不得读取任何 warm/final 搜索统计，只用公开动作执行长期更新：

$$\ell_t^{\text{obs}} = \text{CE}(\text{onehot}(a_t), p(s_t; \phi_t^c, \psi_{t,0}, r = \text{opponent})) .$$

9. 两份长期状态的版本号各自递增；当前手短期状态与全部搜索树销毁。下一手重新从 θ_{search} 创建短期状态。

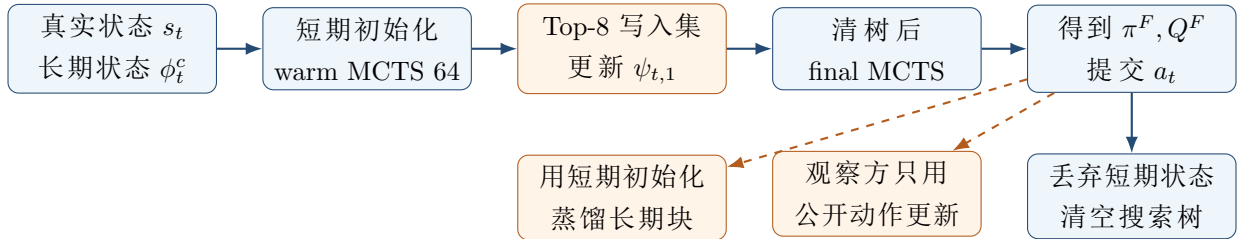


图 1: 双时间尺度真实回合。短期块从 warm tree 学习并服务 final 搜索；长期块只蒸馏 final root，观察方只接收公开动作。

5 训练目标与元学习

5.1 内循环

记 ρ 为 Huber 损失，节点权重为

$$\omega_u = \frac{N_u}{\sum_{v \in \mathcal{D}_t^{\text{warm}}} N_v} .$$

短期写入损失为

$$\ell_t^{\text{search}} = \sum_{u \in \mathcal{D}_t^{\text{warm}}} \omega_u \left[\text{CE}(\pi_u, p_u) + \lambda_q \sum_{a: N_u(a) > 0} \pi_u(a) \rho(q_u(a) - Q_u(a)) + \lambda_s \rho(v_u - V_u) \right],$$

并执行一次无动量 SGD:

$$\psi_{t,1} = \psi_{t,0} - \eta_{\text{search}} \nabla_{\psi_{t,0}} \ell_t^{\text{search}}.$$

多个节点保持 batch size 为 1，顺序累积到同一个平均损失后才更新，不能逐节点改变短期参数。

final 搜索结束并提交真实动作后，以短期初始化重新前向根局面，得到 (p_t^0, q_t^0, v_t^0) 。行棋方长期写入损失为

$$\ell_t^{\text{game,self}} = \text{CE}(\pi_t^F, p_t^0) + \lambda_q \sum_{a: N_t^F(a) > 0} \pi_t^F(a) \rho(q_t^0(a) - Q_t^F(a)) + \lambda_g \rho(v_t^0 - V_t^F),$$

随后只更新 ϕ_t^c :

$$\phi_{t+1}^c = \phi_t^c - \eta_{\text{game}} \nabla_{\phi_t^c} \ell_t^{\text{game,self}}.$$

观察方使用真实动作的 ℓ_t^{obs} 更新 ϕ_t^c ，不产生 Q 或搜索价值损失。 η_{search} 、 η_{game} 及各损失权重都是显式配置；本文档不预设未经实验验证的数值。

训练时使用 `torch.autograd.grad` 并设置 `create_graph=True`，从而保留二阶梯度。运行时参数字典通过 `functional_call` 传入模型[9, 10]。每次内循环都使用参数白名单：短期写入只能更新短期自适应 FFN，长期写入只能更新对应玩家的长期自适应 FFN。

5.2 外循环

在轨迹的每个真实状态上，以适应后的 $\psi_{t,1}$ 评价当前行棋方。记网络在 final 搜索前的输出为 (p_t^+, q_t^+, v_t^+) ，则

$$\mathcal{L}_t = w_t \text{CE}(\pi_t^F, p_t^+) + \lambda_Q \sum_{a: N_t^F(a) > 0} \pi_t^F(a) \rho(q_t^+(a) - Q_t^F(a)) + \lambda_v (z - v_t^+)^2 + \lambda_2 \|\theta\|_2^2.$$

其中 z 是终局结果， w_t 依据 final MCTS 的实际访问数得到。 π_t^F 、 Q_t^F 、 z 、节点选择和动作均停止梯度。当前回合的 $\mathcal{L}_t$ 穿过短期更新，为 θ_{search} 提供“更新后搜索更好”的信用；长期更新是否有效，则通过后续回合中该玩家成为行棋方时的损失获得信用。

外循环更新共享慢权重、 θ_{game} 和 θ_{search} 。一盘棋结束后的 ϕ^B 、 ϕ^W 不写回模型参数；下一条训练序列重新从更新后的长期初始化派生两份状态。短期状态从不跨真实回合进入下一次前向。

6 显存控制、梯度截断与恢复

6.1 16 手截断元梯度

整盘 19 路围棋可能包含数百次长期快权重更新。若保留全部二阶计算图，显存会随手数增长，不适合作为默认实现。项目固定使用 16 个真实回合作为长期元梯度窗口。短期图只覆盖当手的 warm 更新与 final 外层损失，在当手结束后释放，不计入跨手窗口。

在窗口边界保留长期快权重的数值，但截断旧图，并以直通方式重新连接长期初始化：

$$\phi_{\text{next}} = \theta_{\text{game}} + \text{stopgrad}(\phi_{\text{carried}} - \theta_{\text{game}}).$$

该式前向值等于 ϕ_{carried} ，因此长期记忆不会数值重置；反向只传播最近 16 手，并把边界梯度近似路由到长期初始化。 θ_{search} 每手天然重新进入计算图，不使用该重锚定。本文档将长期方法明确标记为截断元梯度近似，而非整盘精确二阶求导。

6.2 Activation Checkpoint

每个 Transformer 块使用 PyTorch 的 `checkpoint` 函数，且显式设置 `use_reentrant=False`。非重入实现会记录前向 autograd 图，支持 `autograd.grad` 和嵌套参数结构，适合本项目的高阶梯度[11]。它通过反向时重算前向降低激活显存，因此预期会增加训练时间。

dropout 固定为零，检查点区域不得依赖会在重算时变化的全局状态。实现完成后必须比较开启和关闭 activation checkpoint 时的策略、标量价值、逐动作 Q、一阶梯度及二阶梯度。

6.3 暂不编译模型

MVP 固定使用 PyTorch eager mode，不启用 `torch.compile`、TorchScript 或 ONNX。原因不是编译在理论上不可行，而是动态参数字典、`create_graph=True`、变长回合、Python 控制流和检查点重算容易引发 graph break、守卫失效或重复编译[12]，会显著增加调试面。

未来若确有需要，只考虑单独编译无 TTT 更新的纯前向推理函数；启用前必须与 eager mode 做策略、标量价值、逐动作 Q、一阶梯度和搜索结果的一致性验证。

6.4 持久化检查点

学习器只在 16 手窗口边界或完整梯度累积边界保存，不序列化 autograd 图。标准训练检查点包含：

- 模型慢权重、 θ_{game} 与 θ_{search} ；
- 外循环优化器与学习率调度器状态；
- Python、NumPy、CPU 和 CUDA 随机数状态；
- 数据游标、全局训练步、已完成累积数；
- 配置文件、代码版本和数据清单的哈希。

若需要恢复一盘未结束的自博弈，另存动作历史、八局面缓存、 ϕ^B 、 ϕ^W 、双方长期版本和随机数状态。短期状态、SearchTrace 与 MCTS 树不进入检查点；保存边界固定在完整真实回合之后。恢复后从当前回合开头重新创建 $\psi_{t,0}$ 并重跑 warm/final 搜索。若训练在 16 手窗口内部中断，恢复时从最近窗口边界重放最多 16 手。

7 监督预训练数据

7.1 数据来源

监督预训练优先采用 KataGo g104 公共自博弈数据。该数据目录在 CC0 下发布，包含拼接 SGF、训练 NPZ、MCTS 策略访问目标、终局结果和棋局标识[7, 8]。它主要采用面积计分，与 MVP 规则方向一致。

数据下载必须使用显式清单记录来源 URL、文件大小、校验和和许可证；不得把数据打包进 Git 或 Docker 镜像。预处理固定执行：

1. 过滤 19×19、面积计分、普通自博弈样本；
2. 按棋局标识和手数排序，并把不连续位置切分为独立序列；
3. 从连续状态重建项目所需的八局面输入；
4. 优先把 MCTS 访问次数归一化为 π_t ，缺失时退化为实际动作 one-hot；
5. 若样本没有逐动作搜索 Q，则显式关闭该样本的 Q 损失，不以终局结果伪造边 Q；
6. 为整条序列采样同一个八重对称变换，确保棋盘历史、动作、访问分布和合法掩码一致；
7. 逐条序列输出，保持 batch size 为 1。

7.2 预训练顺序

每条监督序列开始时，从 θ_{game} 创建黑白两份私有长期状态，然后按真实手顺重放。存在根搜索分布时，行棋方使用该分布更新长期块；观察方始终只使用公开动作。终局结果用于外层价值目标。

监督阶段**不执行短期内循环**，因为现有数据不包含与在线协议匹配的 warm tree 内部节点及边 Q。短期块仍参与普通前向， θ_{search} 仍由外层损失学习一个可用初始化；完整的 warm 更新、更新后评价和逐动作 Q 监督从自博弈阶段启用。不得用同一份离线根标签同时充当 warm 和 final 目标。

允许顺序处理 8 条序列后再执行一次外循环优化器更新，以梯度累积改善稳定性。任意时刻内存中只有当前序列的两份长期运行时状态，不构造多样本 TTT batch。

8 MCTS 与自博弈设置

8.1 搜索基线

MVP 使用 PUCT。每个真实回合先执行固定 64 次、不加 Dirichlet 噪声的 warm MCTS；短期更新并清树后，再执行 final MCTS。final 根节点在自博弈中加入比例 0.25、 $\alpha = 0.03$ 的 Dirichlet 噪声。前 30 手按 final 访问分布采样，之后选择 final 访问次数最大的动作。认输关闭，确保每盘棋产生真实终局目标。

搜索过程必须满足：

- warm 与 final 各自的单次搜索内，长期和短期权重版本固定；
- 非法动作的先验概率为零；
- 叶节点只使用策略 prior 与标量价值；模型逐动作 Q 头不得用于 PUCT 选择、边初始化或备份；
- 备份价值时正确翻转行棋方视角；
- SearchTrace 保存根及筛选内部节点的状态、访问分布、已访问边 Q、节点价值和访问数，并整体停止梯度；
- 短期更新后必须丢弃 warm tree；真实落子和长期更新后必须丢弃 final tree。

8.2 模拟次数随机化

warm 预算恒为额外的 64 次模拟，不从 final 预算中扣除。为降低自博弈平均成本并保留部分高质量策略目标，每个真实回合独立选择 final 搜索预算：

- 75% 的回合使用 128 次快速搜索；
- 25% 的回合使用 800 次完整搜索。

两类回合都执行相同的 warm 更新与长期蒸馏。外循环策略损失按 final 实际访问数相对完整预算加权，避免少量访问产生的尖锐分布与完整搜索拥有相同监督强度。固定 warm 阶段使短期适应数据分布不随 final 档位变化；代价是平均搜索模拟数约增加两成，尚未计入短期反向传播成本。final 随机预算思路参考 KataGo 的 playout cap randomization[5]，但 MVP 不加入其余复杂搜索启发式。

9 工程组织与容器化

建议实现阶段采用以下模块边界：

子系统	职责
规则适配层	不可变围棋状态、合法动作、八局面历史、视角转换与终局结果
模型层	Transformer、策略/标量价值/逐动作 Q 头、两类快权重初始化与函数式前向
记忆控制器	创建黑白长期状态和每手短期状态、参数白名单更新、版本管理与 16 手重锚定
MCTS	PUCT、warm 与 final 两阶段搜索、根噪声、随机 final 预算、SearchTrace 和树生命周期
数据层	g104 下载清单、连续序列重建、对称增强与 batch=1 流式读取
学习器	监督预训练、外循环损失、梯度累积、activation checkpoint 和恢复
自博弈器	双私有长期记忆、单手短期适应、final-root 蒸馏、轨迹写入与未完成棋局快照

Docker 以 Linux、Python 3.12、PyTorch 2.13、CUDA 12.8 和单张 NVIDIA GPU 为参考环境。数据、日志和检查点全部通过 volume 挂载。容器至少提供以下独立命令入口：规则测试、数据预处理、监督预训练、自博弈、AlphaZero 学习器和检查点恢复。

本地 CPU 路径只要求能运行规则测试、数据测试、小模型前向、一阶梯度和小规模二阶梯度，不承诺完成 19 路训练。

10 敏捷开发冲刺

冲刺	主题	完成定义
0	规则与可微性	完成 19 路规则适配、17 平面编码、长期/短期初始化、双长期加单短期状态、标准注意力双反向和非重入 activation checkpoint 最小验证。
1	序列数据	完成 g104 数据清单、许可证记录、连续序列重建、缺失 Q 掩码、整序列对称增强、batch=1 加载和梯度累积；监督阶段不执行短期内循环。

2	标准 AlphaZero	完成策略、标量价值和辅助 Q 输出, PUCT 只消费前两者; 完成根噪声、final 预算随机化、自博弈轨迹和一盘完整 19 路对局。
3	双时间尺度 TTT	完成 warm 64、Top-8 SearchTrace、短期更新、清树 final 搜索、final-root 长期蒸馏、观察方公开动作更新和 16 手长期截断。
4	恢复与容器化	完成学习器检查点、未完成棋局恢复、Docker CPU/GPU 命令及端到端冒烟测试。
未来	非 MVP	前端、后端、棋盘可视化、多 GPU、batch > 1、模型编译、低秩快权重、消融实验、参数搜索和 Elo/性能分析。

11 测试与验收标准

11.1 规则与数据正确性

- 固定棋谱覆盖单子与多子提取、打劫、禁入点、停着、连续双停和终局计分;
- 动作编号、SGF 坐标、策略索引和八重对称变换可逆;
- 八局面输入在提子、停着和开局补零时正确;
- 所有训练样本 batch 维度严格等于 1; 同一序列只使用一个对称变换。

11.2 双初始化、生命周期与隐私

- 模型参数和普通检查点中各有一份 θ_{game} 与 θ_{search} , 不存在按黑白复制的可训练初始化;
- 每盘开始时 ϕ_0^B 与 ϕ_0^W 数值相同, 但存储和后续更新相互独立;
- 每手 $\psi_{t,0}$ 都等于 θ_{search} ; warm 后只有短期块变化, 长期块与慢权重数值不变;
- final 搜索使用 $\psi_{t,1}$, warm tree 已清空, 逐动作 Q 头的任意改变不得影响 PUCT 搜索结果;
- 长期蒸馏使用 $\psi_{t,0}$ 而非 $\psi_{t,1}$, 只改变行棋方长期块, 不复制短期参数;
- 观察方更新结果只依赖公开状态和实际动作; 任意改变对方私有 π_t^F 、 Q_t^F 或 warm trace 均不得影响观察方更新;
- 非法动作与未访问边没有 Q 梯度; 短期状态销毁后, 下一手前向不能引用其存储或 autograd 图;
- 每次状态版本变化后旧树为空, 搜索节点版本与长期/短期参数版本不存在混用。

11.3 梯度、截断与恢复

- 当前回合 final 外层损失对 θ_{search} 产生有限、非零的元梯度；行棋方未来回合损失对 θ_{game} 产生有限、非零的元梯度；
- activation checkpoint 开关前后的策略、标量价值、逐动作 Q、一阶梯度和二阶梯度在规定容差内一致；
- 16 手边界前后长期状态前向数值连续，旧长期 autograd 图已释放；短期图不跨边界；
- warm 写入集只有根或不足 8 个内部节点时仍可完成一次归一化更新；多个节点始终顺序执行 batch-size-1 前向；
- 可完成 batch=1 监督更新和 8 序列梯度累积；
- 学习器恢复后下一窗口结果与无中断运行一致；未完成棋局恢复后不恢复短期状态，并从该回合开头重跑 warm/final 搜索；
- Docker CPU 冒烟测试和单卡 CUDA 冒烟测试通过。

项目最终以机制正确、训练闭环可运行、检查点可恢复和设备可迁移为完成标准，不以 Elo、吞吐量、训练时长或相对基线提升作为验收指标。

12 风险与明确不做的事项

- 二阶显存仍可能过高：先缩小调试模型验证正确性；正式配置使用 16 手截断与逐块 activation checkpoint，不在 MVP 中改成一阶近似。
- 两阶段搜索增加成本：warm 固定额外 64 次且更新后必须清树；该开销是验证单手适应机制的明确代价，不通过复用旧统计降低正确性。
- 搜索自蒸馏可能放大偏差：warm/final 统计全部停止梯度，短期只从较弱 warm 目标更新，当前回合由更强 final 搜索评价；不得让预测 Q 直接进入 PUCT 形成反馈环。
- 内部节点监督可能噪声过高：只选择访问数至少 8 的 Top-8 内部节点，并按访问数加权；不足时退化为根节点，不补造目标。
- 观察方更新可能带来目标冲突：通过 memory_role 显式区分自身策略和对手建模；不通过新增消融实验解决。
- 监督数据与短期协议不匹配：监督阶段不做短期更新；缺失逐动作 Q 时掩码，不用同一根标签伪造 warm/final 对。
- 共享初始化不等于共享运行时记忆：参数量只计算一份长期和一份短期初始化，但对局中仍保留两份已经分化的长期状态及一份当手临时状态。

明确不属于当前范围的内容包括：多 batch 快权重管理、分布式多 GPU、自动超参数搜索、消融实验、性能基准、棋力评级、前端、后端服务、棋盘可视化、模型编译和低秩快权重压缩。

13 推荐实施顺序

首先用小模型完成两个自适应块参数白名单、长期/短期生命周期，以及两条独立元梯度路径：同手 final loss 指向 θ_{search} ，未来回合 loss 指向 θ_{game} 。随后接入完整 19 路 Transformer 与三个输出头，确认辅助 Q 不影响标准 PUCT。

在标准 AlphaZero 回合正确后，先让 MCTS 导出只读 `SearchTrace`，再加入 warm 64、短期更新、强制清树和 final 搜索。短期闭环稳定后，加入使用短期初始化的 final-root 长期蒸馏及观察方公开动作更新。最后接入监督序列、16 手长期截断、持久化检查点和 Docker。

这一顺序把最高风险的四个问题——规则正确性、两类状态的写入边界、搜索缓存失效和高阶梯度可用性——放在项目早期验证，同时保留一个始终可运行的标准 AlphaZero 基线。基线用于工程回退和故障定位，不用于正式消融或性能比较。

参考文献

- [1] Yu Sun, Xinhao Li, Karan Dalal, et al. *Learning to (Learn at Test Time): RNNs with Expressive Hidden States*. 项目本地论文: [paper/Learn at Test Time.pdf](#).
- [2] Arnub Tandon, Karan Dalal, Xinhao Li, et al. *End-to-End Test-Time Training for Long Context*. 项目本地论文: [paper/End-to-End Test-Time Training for Long Context.pdf](#).
- [3] David Silver, Julian Schrittwieser, Karen Simonyan, et al. Mastering the Game of Go without Human Knowledge. *Nature*, 550:354–359, 2017. <https://www.nature.com/articles/nature24270>
- [4] David Silver, Thomas Hubert, Julian Schrittwieser, et al. A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. *Science*, 362:1140–1144, 2018.
- [5] David J. Wu. Accelerating Self-Play Learning in Go. arXiv:1902.10565, 2020 revision. <https://arxiv.org/abs/1902.10565>
- [6] Farama Foundation. PettingZoo Go Environment, go_v5. https://pettingzoo.farama.org/main/_modules/pettingzoo/classic/go/go/
- [7] KataGo Project. g104 Downloadable Self-Play Data: Format and Contents. <https://katagoarchive.org/g104/README.txt>
- [8] KataGo Project and Jane Street. g104 Data License (CC0). <https://katagoarchive.org/g104/LICENSE.txt>
- [9] PyTorch Contributors. `torch.func.functional_call` Documentation. https://docs.pytorch.org/docs/main/generated/torch.func.functional_call.html
- [10] PyTorch Contributors. `torch.autograd.grad` Documentation. <https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html>
- [11] PyTorch Contributors. Activation Checkpointing Documentation. <https://docs.pytorch.org/docs/stable/checkpoint.html>
- [12] PyTorch Contributors. `torch.compile` Troubleshooting and Graph Breaks. https://docs.pytorch.org/docs/main/user_guide/torch_compiler/torch_compiler_troubleshooting.html